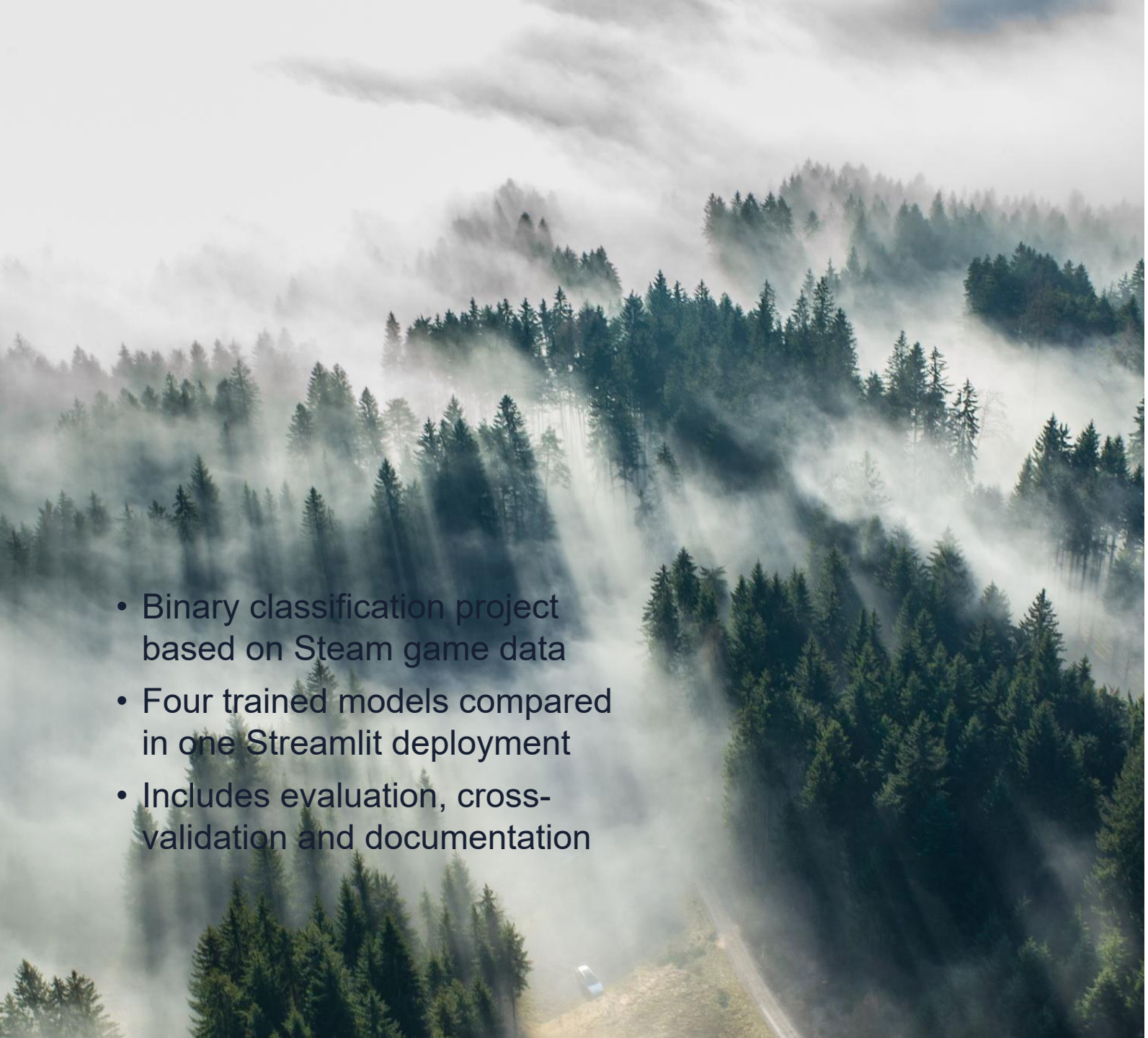


- Binary classification project based on Steam game data
- Four trained models compared in one Streamlit deployment
- Includes evaluation, cross-validation and documentation

Steam Game Success Predictor

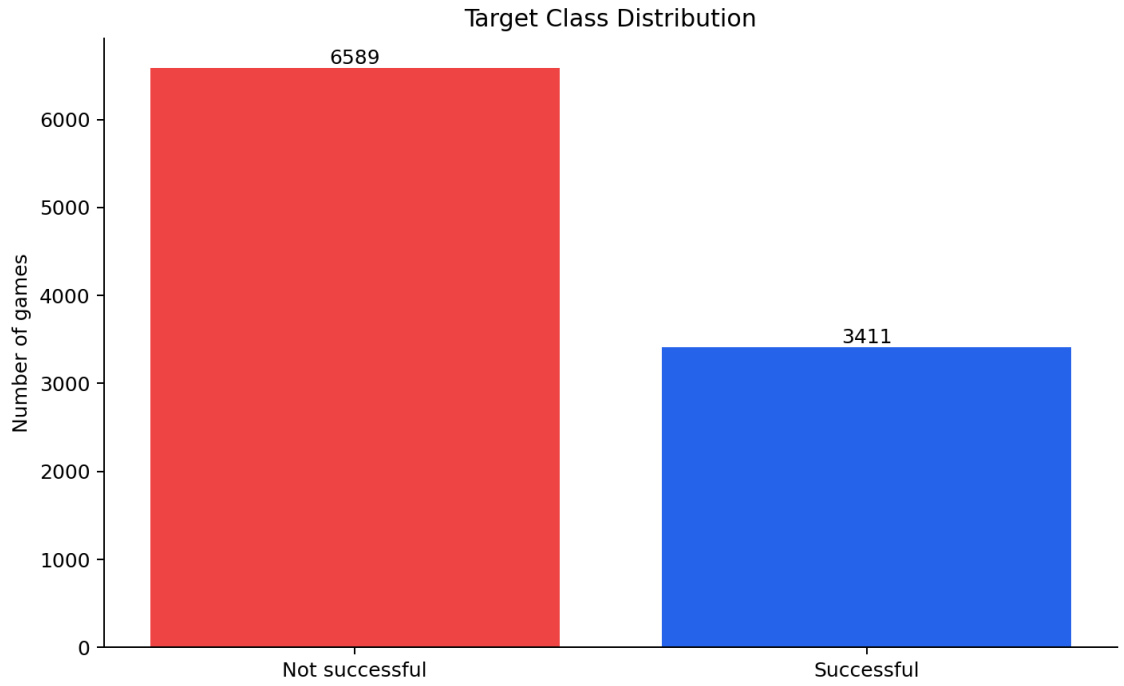
- KNN, SVM, Random Forest and Deep Learning

- Author: Mussin Yesbol
- Professor: Piotr Wozniak

- Module: Machine learning in Python

Project Goal

- Predict whether a Steam game is successful or not successful
- Use numerical features such as reviews, playtime, price and CCU
- Compare classical ML models with a neural network
- Deploy all models in a single interactive application



Dataset and Target Variable

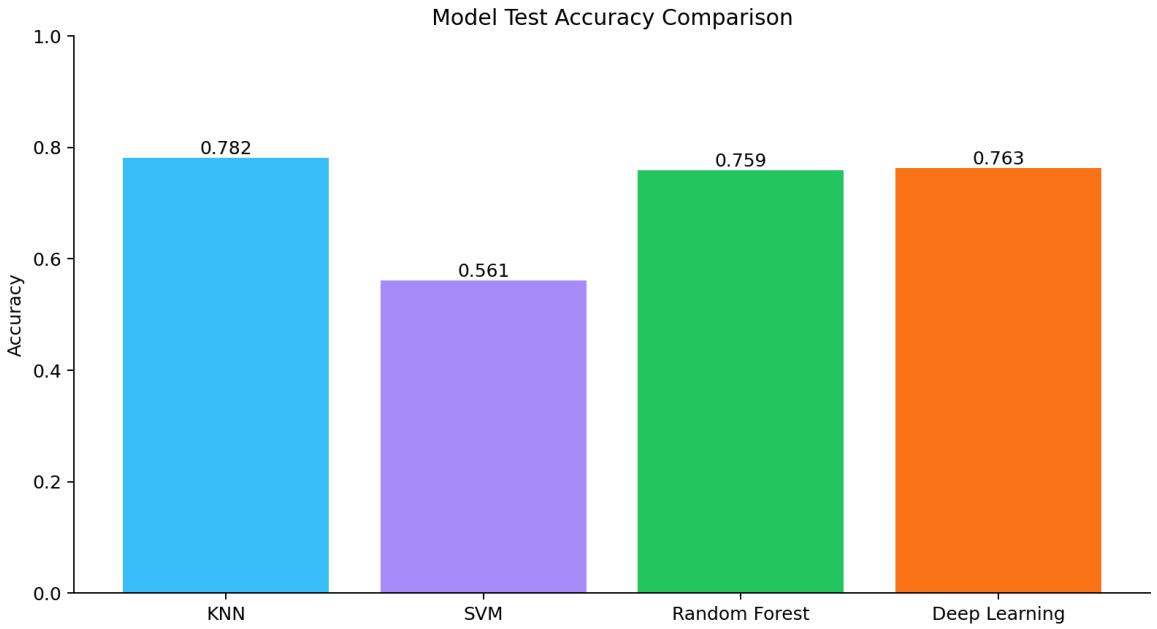
- Target variable: success
- Success is based on rating ratio and concurrent user activity
- Leakage columns are removed from improved models

Preprocessing in Jupyter

Jupyter Notebook - Target creation and leakage prevention

```
df["rating_ratio"] = df["positive"] / (df["positive"] + df["negative"] + 1)

df["success"] = (
    (df["rating_ratio"] >= 0.8) &
    (df["ccu"] > df["ccu"].median())
).astype(int)

columns_to_drop = [
    "appid", "name", "developer", "publisher",
    "score_rank", "owners",
    "positive", "negative", "rating_ratio", "ccu"
]

# Remove leakage columns before training.
df = df.drop(columns=[col for col in columns_to_drop if col in df.columns])
```

- Create rating_ratio and success
- Remove text columns and leakage-related columns
- Prepare final feature matrix for model training



Implemented Models

- KNN: distance-based baseline model
- SVM: RBF kernel with balanced class weights
- Random Forest: tree ensemble for tabular data
- Deep Learning: dense neural network with sigmoid output

KNN Notebook - Scaling, training and evaluation

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Random Forest Notebook - Model training and confusion matrix

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight="balanced"
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

Classical ML Notebook Code

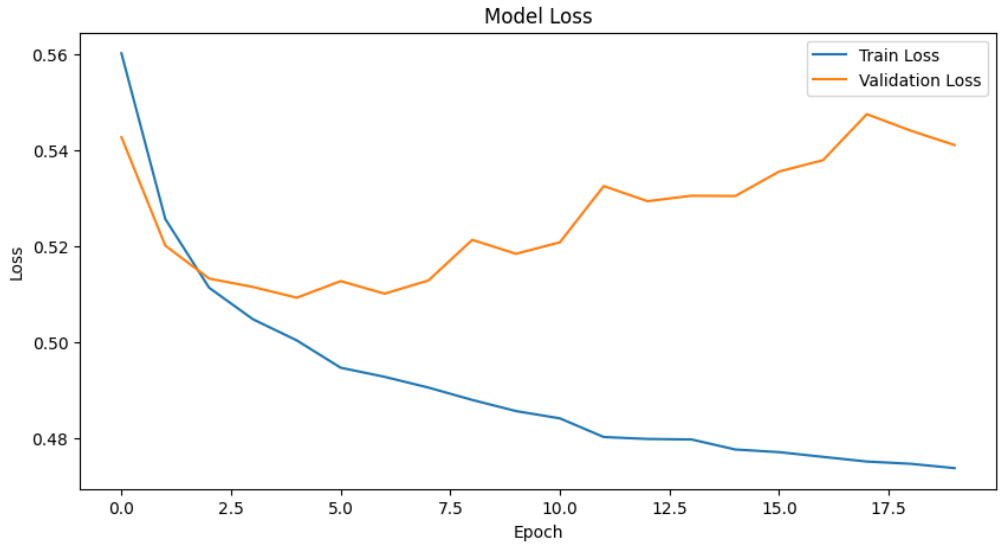
- KNN and SVM use StandardScaler
- Random Forest uses class_weight='balanced'
- Models are evaluated with accuracy, classification report and confusion matrix

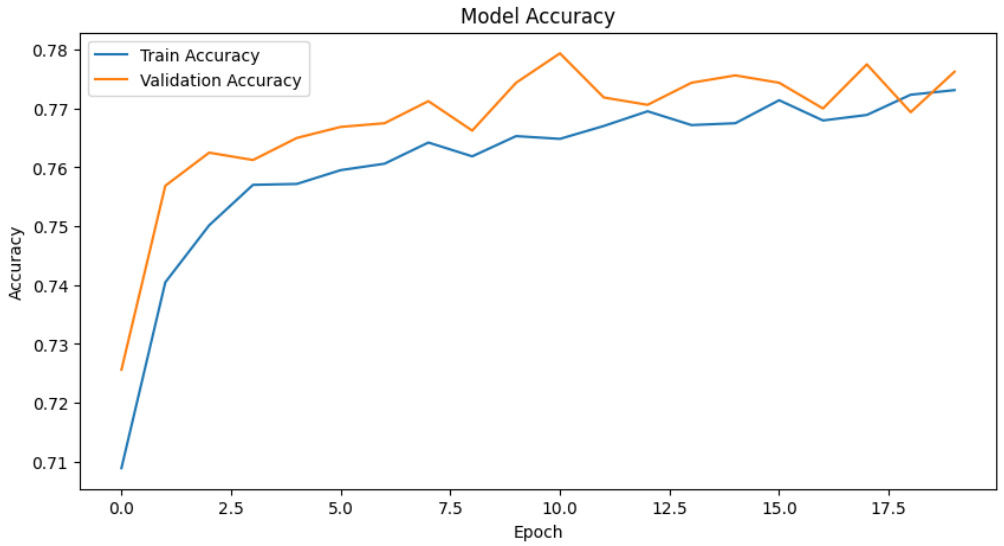
Deep Learning Model

Deep Learning Notebook - Neural network architecture and training

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(64, activation="relu", input_shape=(X_train.shape[1],)),
    Dense(32, activation="relu"),
    Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2
)
```

- Neural network built with TensorFlow/Keras
- Dense layers with ReLU activations
- Sigmoid output for binary classification





Deep Learning Training Curves

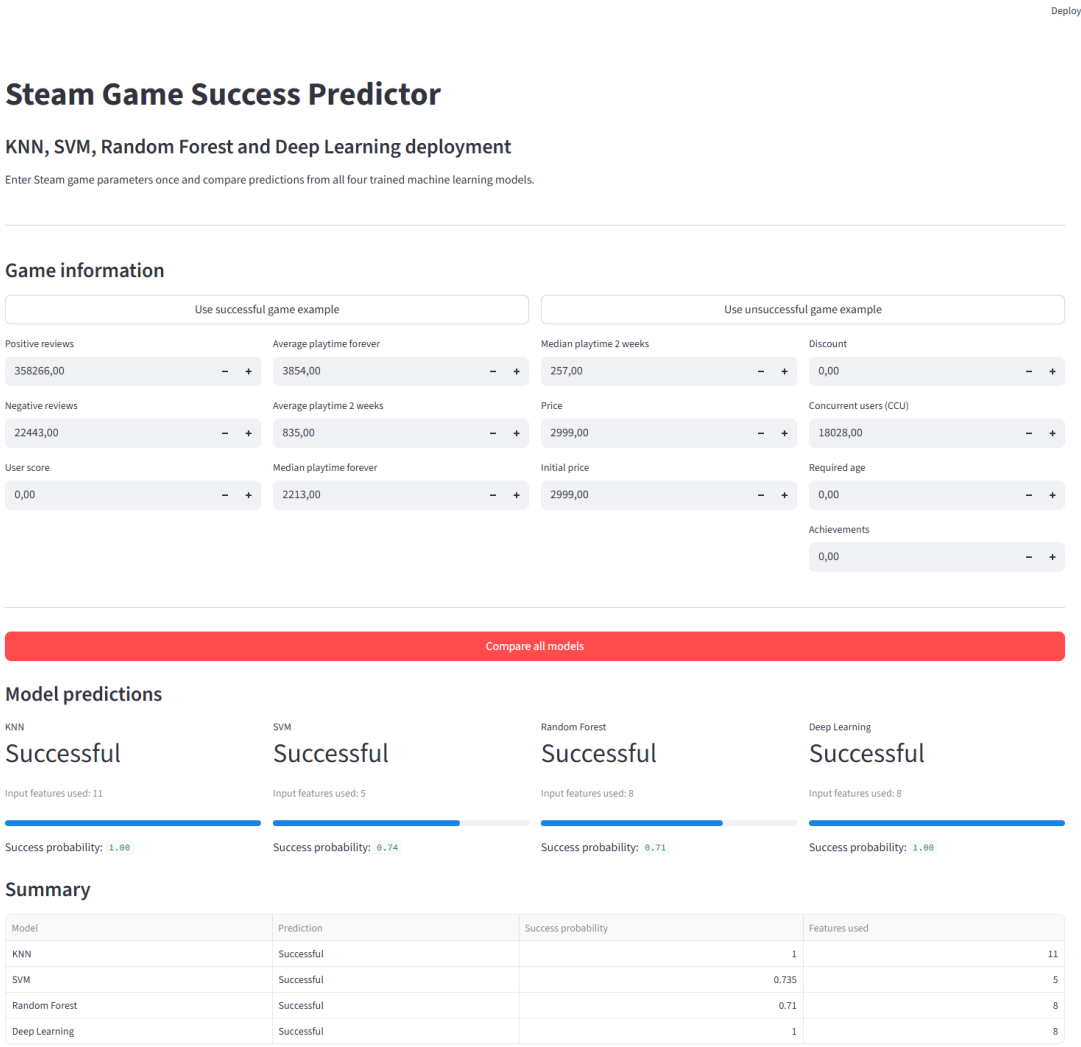
- Accuracy increases during training
- Validation loss suggests possible later overfitting
- Final test accuracy is approximately 0.763

Cross-Validation

Notebooks - Cross-validation with mean and standard deviation

```
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import Pipeline

cv_model = Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier(n_neighbors=5))
])

cv_scores = cross_val_score(cv_model, X, y, cv=5, scoring="accuracy")

print("Cross-validation scores:", cv_scores)
print("Mean accuracy:", cv_scores.mean())
print("Standard deviation:", cv_scores.std())

# Random Forest uses the same cross_val_score idea, but without scaler.
```

- Added to KNN, SVM and Random Forest notebooks
- Mean accuracy shows average model performance
- Standard deviation shows model stability between folds
- KNN and SVM use Pipeline to avoid scaling leakage



Combined Streamlit Deployment

- Root app.py loads all four saved models
- One input form is shared by all models
- Results are shown side by side with probability or score

Deployment Code

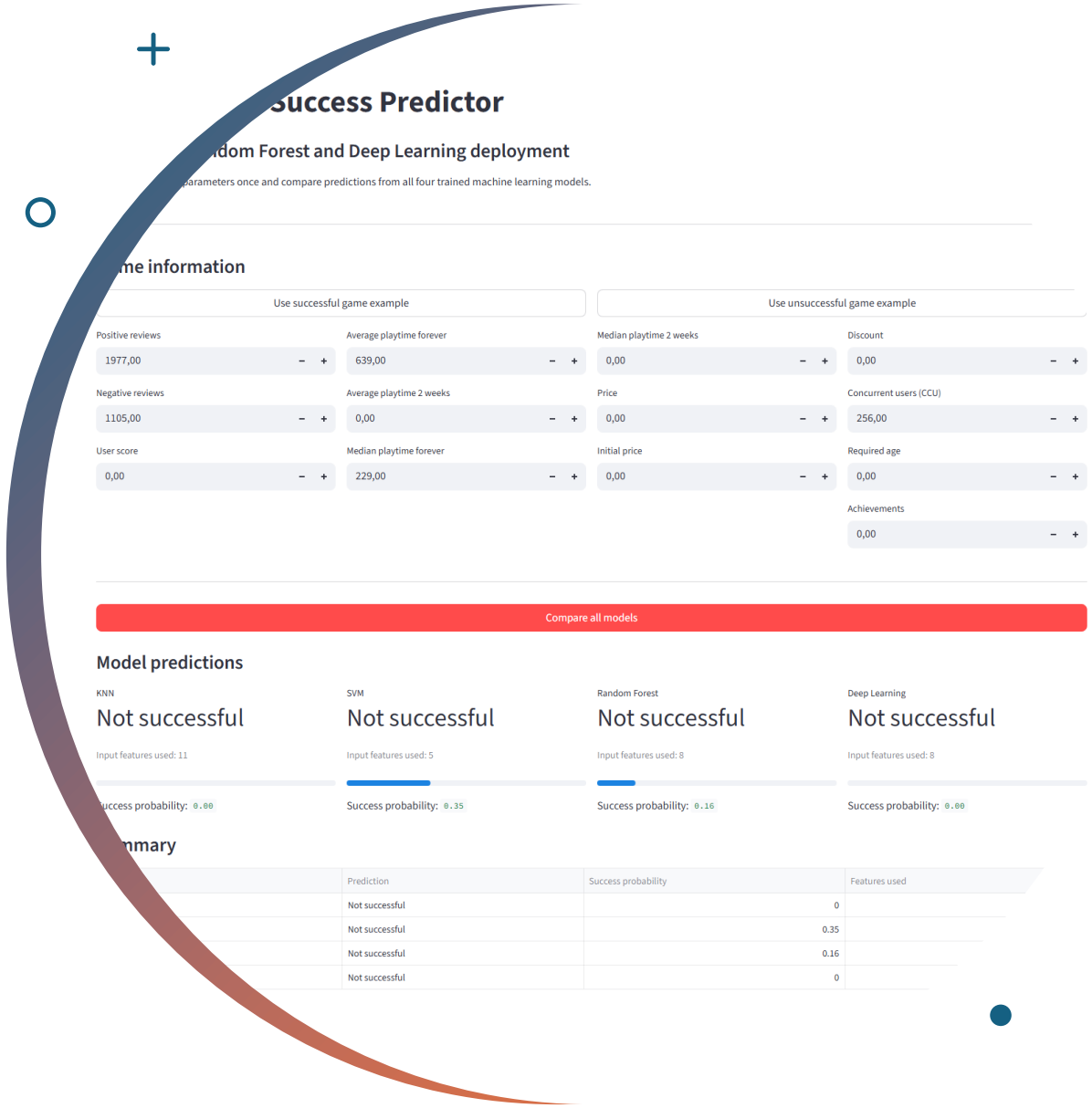
- Models and scalers are loaded from project folders
- SVM uses decision_function because it was trained without probability=True
- Preset buttons fill examples for successful and unsuccessful games

app.py - Loading all saved model artifacts

```python
@st.cache_resource
def load_sklearn_artifacts():
    return {
        "KNN": {
            "model": joblib.load(BASE_DIR / "KNN_Project" / "knn_model.pkl"),
            "scaler": joblib.load(BASE_DIR / "KNN_Project" / "scaler.pkl"),
            "features": KNN_FEATURES,
            "uses_scaler": True,
        },
        "SVM": {
            "model": joblib.load(BASE_DIR / "SVM_Project" / "svm_model.pkl"),
            "scaler": joblib.load(BASE_DIR / "SVM_Project" / "scaler.pkl"),
            "features": joblib.load(BASE_DIR / "SVM_Project" / "svm_features.pkl"),
            "uses_scaler": True,
        },
        "Random Forest": {
            "model": joblib.load(BASE_DIR / "randomForest" / "random_forest_model.pkl"),
            "features": joblib.load(BASE_DIR / "randomForest" / "rf_features.pkl"),
            "uses_scaler": False,
        },
    }
```

app.py - Prediction and probability logic

```python
def predict_sklearn(name, artifacts, values):
    input_data = build_input_frame(values, artifacts["features"])

    if artifacts["uses_scaler"]:
        input_data = artifacts["scaler"].transform(input_data)

    prediction = int(artifacts["model"].predict(input_data)[0])
    probability = None

    if hasattr(artifacts["model"], "predict_proba"):
        probability = float(artifacts["model"].predict_proba(input_data)[0][1])
    elif hasattr(artifacts["model"], "decision_function"):
        # SVM was trained without probability=True, so use decision_function.
        decision_score = float(artifacts["model"].decision_function(input_data)[0])
        probability = sigmoid(decision_score)

    return {
        "model": name,
        "prediction": prediction,
        "probability": probability,
        "features": len(artifacts["features"]),
    }
```



Limitations and Future Work

- Success definition is an approximation, not real revenue
- SVM probability should be calibrated by retraining with probability=True
- Future work: GridSearchCV, ROC-AUC, more features, online deployment
- Project demonstrates the full ML pipeline from data to deployment

Thank You

- It was pleasure for me working with nice people of RzIT and make a lot of projects together
- P.S. Besides this project i really liked working on the PerfMon project and robot **ALF I** (Alf Pierwszy)

Video, can be seen after downloading the file from google drive

RZESZOWSKA GRUPA IT